Universidade Federal de Campina Grande

Unidade Acadêmica de Sistemas e Computação
Laboratório de Programação 2
Professores: Eliane Araújo e Lívia Sampaio e Melina Mongiovi

Simulado Prova 1

Aluno(a): _____ mat:_____

A startup **RepareBem** foi criada para realizar ajustes em roupas, mantendo a sua qualidade de fábrica, estimulando o consumo consciente, evitando descarte e compras desnecessárias. Ela está informatizando os seus serviços vislumbrando um crescimento vertiginoso neste segundo semestre do ano. Você deve **criar um sistema** que registra e controla os serviços **da loja RepareBem** de acordo com o que se pede.

Não é necessário implementar o Main, a classe que interage com o usuário, mas sim, a classe de sistema **RepareBem,** que fornece as funcionalidades que seriam invocadas pelo Main.

🧵🪡 Os reparos executados nas roupas podem ser, por exemplo, substituição de zíper, barra de calça, apertar cintura, costura de rasgão, etc. Os reparos são cadastrados no sistema, com um identificador textual, descrição e preço. Não devem ser cadastrados reparos com o mesmo identificador, assim, dois reparos são iguais se tiverem o mesmo identificador textual. O sistema deve permitir o reajuste percentual dos preços dos reparos.

🙂👚👖 Os serviços contratados pelos clientes na **RepareBem** são organizados em ordens de serviço - OS. Elas devem ser cadastradas no sistema e identificadas sequencialmente: A primeira OS tem identificador id=1, a segunda id=2, etc. Além disso, elas devem ter a informação do nome e telefone do cliente e o nome da peça de roupa que será reparada. Cada OS refere-se aos reparos feitos em uma peça de roupa. O preço total de um serviço será o somatório de todos os reparos executados naquela roupa. Há um limite de 10 reparos por OS. A representação textual de uma OS deve ter: id, cliente, a roupa, a quantidade de reparos e o seu preço total. Exemplo:

#1; cliente: Maria Helena; roupa: calça; reparos: 1; total: R$20,00

🧑‍🏭📊O sistema deve fornecer uma visão geral das ordens de serviço de acordo com os status possíveis: "Concluída" ou "Em andamento" ou "Não iniciada". Para tanto, deve ter a funcionalidade de alterar o status das OS e listá-las. Considere que quando uma OS é criada, o seu status é "Não iniciada". Deverá ser possível listar todas as OS ou filtrá-las de acordo com o status que se deseja visualizar.

Uma representação de saída da listagem de OS com o filtro de status "Concluída" seria:

Ordens de Serviço - Concluída *#1; cliente: Maria Helena; roupa: calça; reparos: 1; total: R$20,00* *#4; cliente: Gabriel; roupa: bermuda; reparos: 2; total: R$40,00*

Caso não haja OS a serem listadas com o status requerido, deve imprimir a mensagem: *"Não há ordens de serviço do tipo <status>".*

A classe **RepareBem** seria invocada pelo Main para realizar as ações especificadas para o sistema. Essa classe deve ter métodos para implementar as funcionalidades seguintes:

- cadastrarReparo(idReparo:String, descricao:String, preco:double):void;
- reajustarPrecoReparo(idReparo:String, percentual:double):void
- cadastrarOrdemDeServico(cliente:String, telefone:String, roupa:String):int
- exibirOrdemDeServico(idOS:int):String
- incluirReparoOrdemDeServico(idOS:int, idReparo:String): void
- mudarStatusOrdemDeServico(idOS:int, status:String): void
- obterValorOrdemServico(idOS:int):double
- listarOrdemDeServico(status:String): String
- listarOrdensDeServico( ):String //Serão listadas todas as OS cadastradas

🚫O sistema deve **parar**, caso seja fornecido um identificador de Ordem de Serviço ou Reparo inexistentes nas operações que necessitem desse argumento. Para viabilizar este comportamento, lance a **exceção IllegalArgumentException** de Java.

🟢🔴Faça testes de unidade para as funcionalidades de **reajustarPrecoReparo** na classe **Reparo** e **mudarStatus** na classe **OrdemDeServico.** Não precisa testar estas funcionalidades na classe de sistema.

LEMBRE-SE: Não é necessário implementar Main. Use o Junit para exercitar o seu programa!